ASSIGNMENT 5 - PART B

B2 - CONTAINERISE THE PORTAL

Implementation and Experimental Investigation | 5 Marks | K5 - Evaluate

1. Objective

The task requires a Dockerfile for the campus result-portal Flask application, a successful image build and port mapping with docker run, a Docker Compose deployment with a database volume, CPU/memory constraints demonstrated using docker stats, and inspection/optimisation of image layers. The three supplied Dockerfile TODOs must be completed.

2. Application and Dockerfile implementation

A small Flask app is included with a home route and a health-check route. When database environment variables are present, it checks connectivity to PostgreSQL. When run standalone without a database, it still serves the portal and reports that the database is not configured.

Completed Dockerfile

```
FROM python:3.12-slim

# TODO 1
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Create the non-root portal account.
RUN groupadd --system portal \
    && useradd --system --gid portal --create-home \
       --home-dir /home/portal --shell /usr/sbin/nologin portal

# TODO 2
COPY --chown=portal:portal app.py /app/app.py
USER portal

# TODO 3
EXPOSE 8000
CMD ["gunicorn", "--workers", "2", "--bind", "0.0.0.0:8000", "app:app"]
```

TODO 1 sets the image working directory to `/app`. Dependency installation occurs before source-code copying so Docker can reuse the dependency layer when only `app.py` changes. TODO 2 copies the application and switches to a dedicated non-root user named `portal`. TODO 3 documents port 8000 and launches Gunicorn on all container interfaces.

3. Build and run with docker run

```
docker build -t result-portal .
docker image ls result-portal

docker run -d --name result-portal-standalone \
  -p 8000:8000 --cpus="0.50" --memory="512m" \
  result-portal

docker ps
curl -i http://localhost:8000/
curl -i http://localhost:8000/healthz
docker exec result-portal-standalone whoami
docker stats --no-stream result-portal-standalone
```

The `-p 8000:8000` option publishes host TCP port 8000 and forwards it to container TCP port 8000. `--cpus` and `--memory` impose resource ceilings. The required identity check should print `portal`, proving the application is not running as root. The actual image size and live resource values must be recorded from the student's Docker environment.

4. Docker Compose with persistent database data

The supplied Compose file defines a portal service and a PostgreSQL service on a private bridge network. The portal waits for the database health check before starting. The database data directory is mounted on a named volume, so database files survive container replacement.

```
cp .env.example .env
# Edit .env and replace the example password.

docker compose up --build -d
docker compose ps
docker compose logs --tail=50
curl -i http://localhost:8000/
docker compose exec portal whoami
docker stats --no-stream
```

The named volume `postgres_data` is mounted at `/var/lib/postgresql/data`. To demonstrate persistence, run `docker compose down` followed by `docker compose up -d`. Do not use `docker compose down -v` during this test because `-v` removes named volumes.

5. CPU/memory limits and docker stats

Control	Standalone docker run	Compose example
CPU	`--cpus="0.50"`	Portal: 0.50 CPU; database: 1.00 CPU
Memory	`--memory="512m"`	Portal: 512 MiB; database: 1 GiB
Observation	`docker stats --no-stream` or `docker stats`	`docker stats --no-stream` for all Compose containers

Docker statistics report current CPU, memory, network and block I/O usage. A configured maximum is not the same as actual consumption: a lightly loaded container may use far less than its limit. Record the output before and during a controlled request test; do not force memory exhaustion on a shared or production host.

6. Image layers and optimisation

```
docker history result-portal:latest
docker image inspect result-portal:latest
docker images result-portal
```

The Dockerfile copies and installs `requirements.txt` before copying `app.py`. This ordering separates dependency changes from application-source changes and supports layer caching. The `python:3.12-slim` base omits many packages and tools present in a full general-purpose distribution. The `.dockerignore` file excludes local virtual environments, logs, caches, secrets and evidence files from the build context.

The package also contains `Dockerfile.multistage`, which installs dependencies in a builder stage and copies the installed files into a clean runtime stage, and `Dockerfile.ubuntu`, an optional full-Ubuntu comparison. They can be built with:

```
docker build -f Dockerfile.multistage -t result-portal:multistage .
docker build -f Dockerfile.ubuntu -t result-portal:ubuntu .
docker images result-portal
```

A multi-stage build is most useful when build tools are required but not needed at runtime. In this small app, the slim image may already be compact and binary wheels can reduce the benefit of an extra builder

stage. Compare actual image sizes rather than assuming a fixed saving.

7. Alternative concept - full Ubuntu base or multi-stage

Measure	Full Ubuntu base	python:3.12-slim / multi-stage
Image size	Usually larger because the base contains more general-purpose OS packages and utilities.	Usually smaller because the base is focused on the Python runtime; multi-stage can omit build-only tools.
Build time	May take longer when more OS packages must be installed and updated.	Often quicker for a small Python app; multi-stage can add a build stage but keep final runtime lean.
Pull time	Larger layers require more bytes to transfer when not cached.	Smaller layers generally pull faster on the same network and cache state.
Security/package exposure	A broader installed package set can contain more components requiring patching and vulnerability review.	Fewer installed packages generally reduce the component inventory, but application dependencies still need scanning and updates.

Pull-time estimate: transfer time in seconds is approximately image size in MB multiplied by 8 and divided by available throughput in Mbps. Actual time also includes registry latency, protocol overhead, decompression, layer verification, caching and contention. The number of packages with vulnerabilities cannot be inferred from the base image name; it must be measured with a scanner against the image built at the time of testing.

If Trivy is installed, scan both images and record the actual findings:

```
trivy image --scanners vuln result-portal:latest
trivy image --scanners vuln result-portal:ubuntu
```

Report the number of affected packages and findings by severity. A count of CVEs is not the same as a count of vulnerable packages, so state exactly which metric is used.

8. Experimental evidence to collect

Evidence	Command / expected result
Successful build	`docker build -t result-portal .` completes without errors.
Running container	`docker ps` lists the portal container as running.
Browser/API access	`http://localhost:8000` displays the portal; `/healthz` returns status ok.
Non-root user	`docker exec <container> whoami` prints `portal`.
Image size	Record the displayed size from `docker images result-portal`.
CPU/memory	Record actual values from `docker stats` with the configured limits.
Persistent DB data	Compose named volume remains after `docker compose down` without `-v` and services restart.
Layer history	`docker history result-portal` displays the image instruction/layer history.

The report does not claim the image build, Compose deployment, measured size, pull time, resource usage or vulnerability scan has run on the student's Docker host. These are execution-time evidence required by the question and are provided as fields in the measurement worksheet.

9. Conclusion

The Dockerfile completes all three TODOs using the required Python slim base, places the app in `/app`, runs Gunicorn as non-root `portal`, and exposes port 8000. Docker Compose runs the portal with PostgreSQL and persists database files in a named volume. Direct Docker resource flags and Compose resource settings constrain CPU/memory, while `docker stats` provides runtime evidence. `docker history` and the alternative Ubuntu/multi-stage builds support the image-layer and optimisation evaluation. Final size, pull-time, package and resource figures must be measured on the Docker environment used for submission.

10. Execution Procedure and Interpretation

Run these steps from the project directory containing Dockerfile, app.py, requirements.txt, compose.yaml and .env.example. Docker Engine must be running. Record command output from your own machine.

10.1 Build and run the standalone portal

```
cp .env.example .env
# Change the sample password before Compose use.
docker build -t result-portal .
docker image ls result-portal
docker run -d --name result-portal-standalone \\
  -p 8000:8000 --cpus=0.50 --memory=512m result-portal
curl -i http://localhost:8000/
docker exec result-portal-standalone whoami
```

10.2 Start the portal and database with Compose

```
docker compose up --build -d
docker compose ps
docker compose logs --tail=50
curl -i http://localhost:8000/
docker compose exec portal whoami
docker stats --no-stream
```

10.3 Verify persistence and inspect image layers

```
docker compose down
docker compose up -d
docker history result-portal:latest
docker image inspect result-portal:latest
docker images result-portal
```

The database volume is retained when Compose is brought down without -v. After the stack restarts, check that PostgreSQL is healthy. Do not use docker compose down -v during the persistence test because that removes the named volume and its stored database files.

10.4 Interpretation checklist

- docker ps confirms the container is running; curl confirms that the published port reaches Flask.
- docker exec ... whoami should print portal, not root.
- docker stats reports sampled CPU and memory use; compare actual use with configured limits.
- docker history shows build instructions/layers; EXPOSE and CMD are image metadata.
- docker images gives the local image size. Record it rather than guessing.
- Compare build/pull times on the same machine/network; state cache condition and throughput.

11. Technical References

[1] Docker Docs, Dockerfile reference: https://docs.docker.com/reference/dockerfile/
[2] Docker Docs, Compose services and resource limits: https://docs.docker.com/reference/compose-file/services/
[3] Docker Docs, image layers and build cache: https://docs.docker.com/learn/content/how-image-layers-and-the-build-cache-work/
[4] Docker Docs, container resource constraints: https://docs.docker.com/engine/containers/resource_constraints/
[5] Docker Docs, image history: https://docs.docker.com/reference/cli/docker/image/history/
Source: Assignment 5 - Part B - Task B2, 5 Marks, page 47. Task alignment includes all three Dockerfile TODOs, docker run, Compose database volume, limits, docker stats and image-history/size evaluation.